

Especificación Técnica de Ingeniería

React Todo Application

Ingeniería de Software - LJCR

15 de abril de 2026

Resumen

Este documento detalla la arquitectura, el diseño y la implementación técnica de una aplicación de gestión de tareas (TODO) desarrollada con React 18. Se analiza el uso de patrones de estado global mediante la Context API, la persistencia de datos asíncrona y la modularización de componentes siguiendo los principios de alta cohesión y bajo acoplamiento.

Índice

1. Introducción

La aplicación React Todo es un sistema reactivo diseñado para la gestión eficiente de tareas personales. El núcleo de la aplicación se basa en un paradigma funcional, utilizando *Hooks* personalizados para la lógica de negocio y la API de Contexto para la distribución del estado sin incurrir en *Prop Drilling*.

2. Arquitectura del Sistema

La arquitectura se divide en tres capas principales que garantizan la mantenibilidad del código:

2.1. Capa de Persistencia (Custom Hooks)

Se implementa el patrón de adaptador para el almacenamiento local mediante el hook `useLocalStorage`. Este encapsula la lógica de sincronización con el navegador y maneja estados asíncronos simulados.

2.2. Capa de Estado Global (Context API)

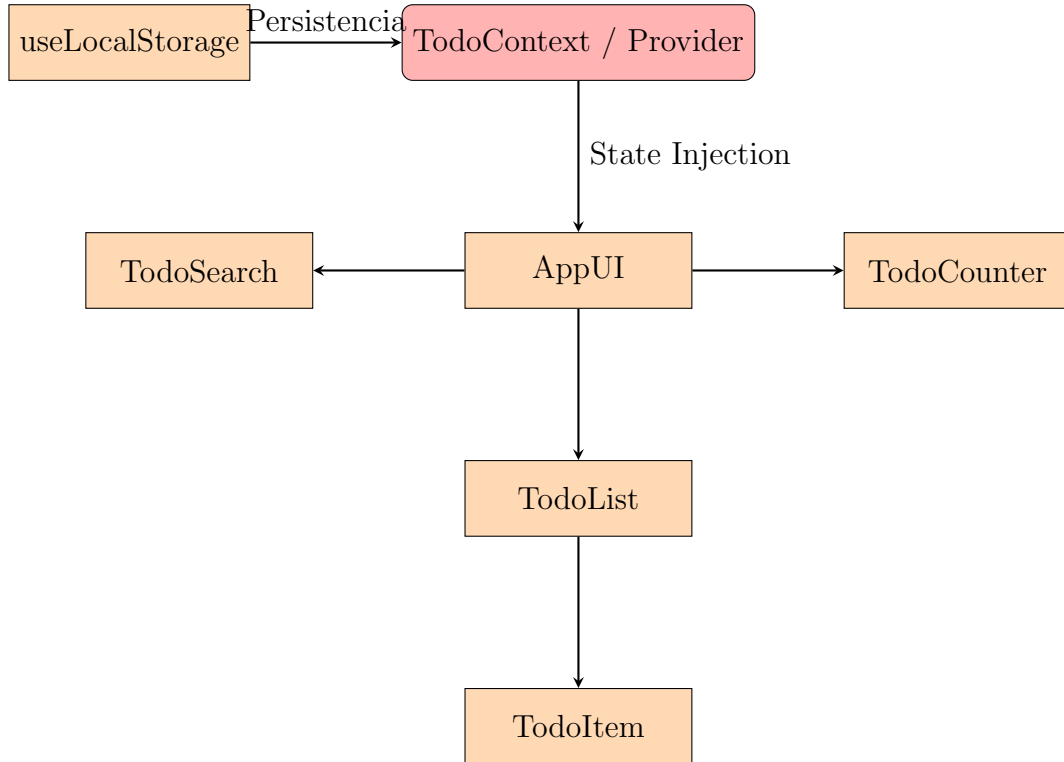
El `TodoProvider` actúa como el orquestador central. Gestiona el estado de los ítems, la lógica de filtrado y el control de modales, exponiendo una interfaz unificada a los componentes consumidores.

2.3. Capa de Interfaz (Componentes)

Componentes atómicos y funcionales que consumen el contexto. Se aplica la convención de un archivo CSS por componente para evitar colisiones de estilos.

3. Diagrama de Arquitectura

A continuación se presenta la jerarquía de componentes y el flujo del estado global:



4. Detalle Técnico de Implementación

4.1. Gestión de Estado Persistente

El hook `useLocalStorage` utiliza `useEffect` para la inicialización y `useCallback` para optimizar la función de guardado:

```

1 const useLocalStorage = (storageKey, initialData) => {
2   const [storedData, setStoredData] = React.useState(initialData);
3   // ... logica de carga
4   const saveData = React.useCallback((newData) => {
5     localStorage.setItem(storageKey, JSON.stringify(newData));
6     setStoredData(newData);
7   }, [storageKey]);
8   return { storedData, saveData };
9 };

```

Listing 1: Fragmento de `useLocalStorage.js`

4.2. Lógica de Filtrado (Memoización)

Para optimizar el rendimiento, se utiliza `useMemo` para el filtrado de tareas, evitando recálculos innecesarios durante re-renderizados causados por otros estados:

$$T_{searched} = \{t \in T_{todos} \mid t.text \text{ contiene } S_{search}\} \quad (1)$$

5. Principios de Ingeniería Aplicados

- **Single Responsibility Principle (SRP):** Cada componente tiene una única razón para cambiar. Por ejemplo, `TodoSearch` solo maneja la entrada de usuario.
- **Separation of Concerns:** La lógica de persistencia está totalmente desacoplada de la interfaz de usuario.
- **Declarative UI:** La interfaz se describe en función del estado, no de transiciones manuales del DOM.

6. Conclusión

La arquitectura propuesta permite escalar la aplicación de manera sencilla. El uso de LaTeX para esta documentación asegura que las especificaciones técnicas se mantengan con un estándar de calidad profesional, facilitando la transferencia de conocimiento entre ingenieros.